

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет Программной Инженерии и Компьютерной Техники

Системы компьютерной обработки изображений

Лабораторная работа № 4

«Формирование распределения случайных величин в заданной области
определения»

Выполнили

Баянов Равиль Динарович

Кузнецов Даниил Александрович

Группа № Р3434

Преподаватель: Жданов Дмитрий Дмитриевич

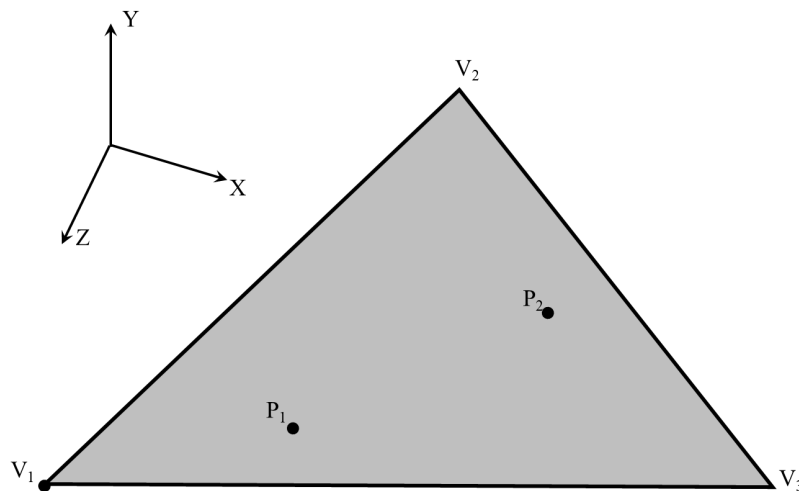
г. Санкт-Петербург 2025

Цель

Изучить методы формирования заданных распределений случайных координат и направлений лучей в пространстве. Доказать корректность полученных распределений случайных величин.

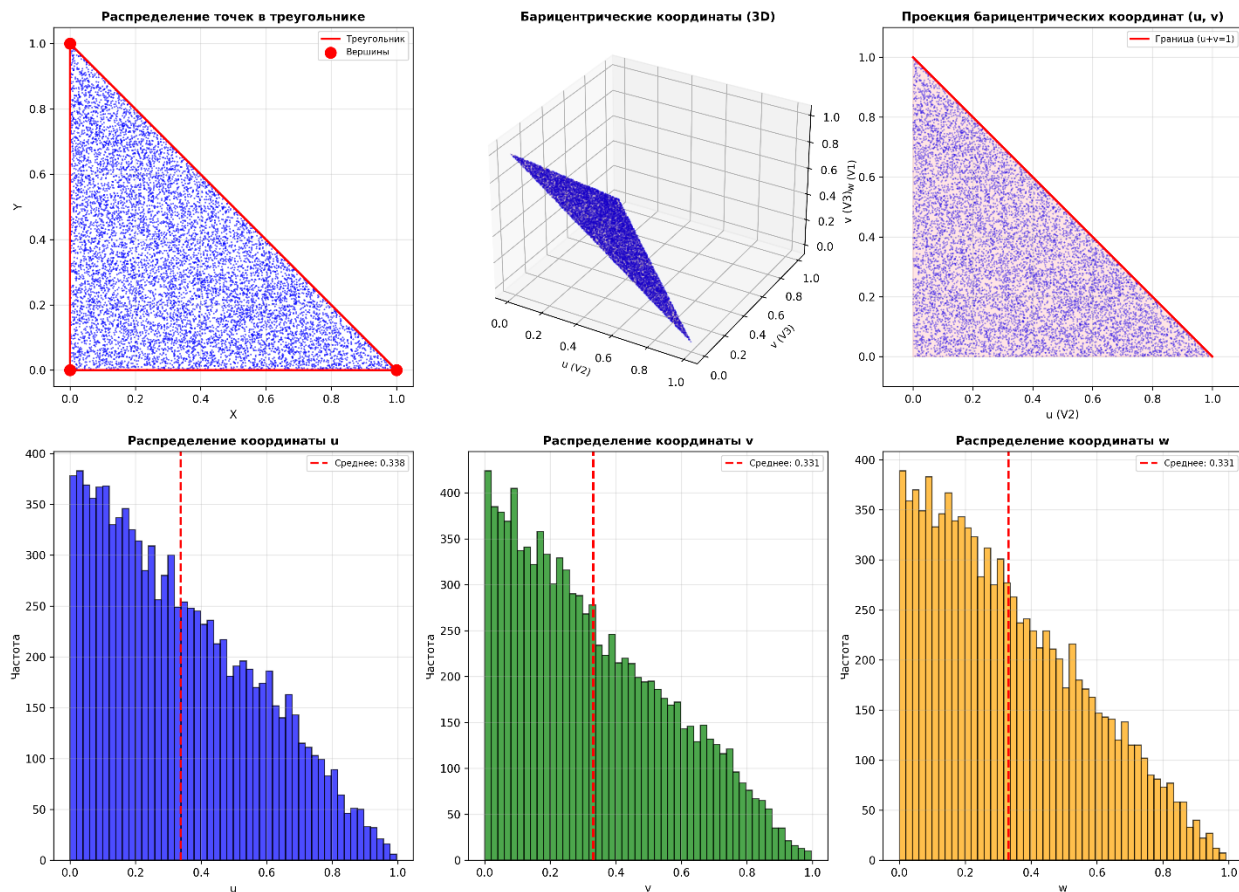
Задание

1. Сформировать равномерное распределение случайных точек ($P_1, P_2, \dots$) внутри треугольника. Треугольник задан тремя координатами в пространстве (V_1, V_2, V_3). Число точек используемых в выборке 100000, число точек, сформированных внутри треугольника также 100000.

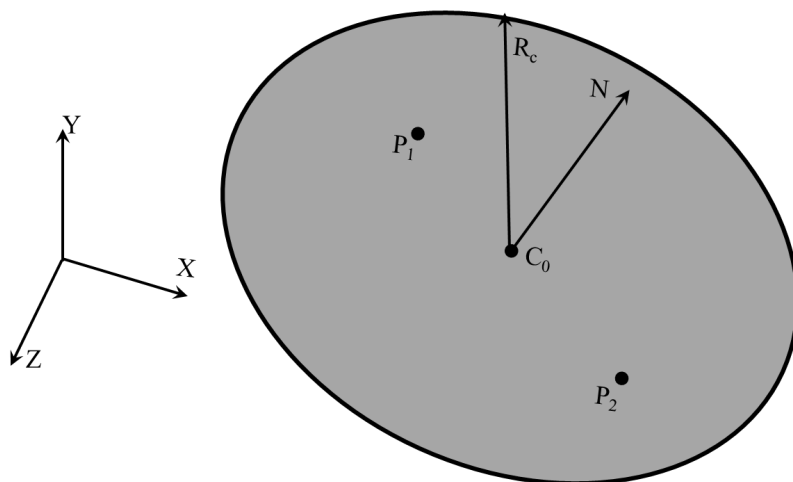


Доказать, что полученное распределение равномерное и все точки лежат внутри треугольника.

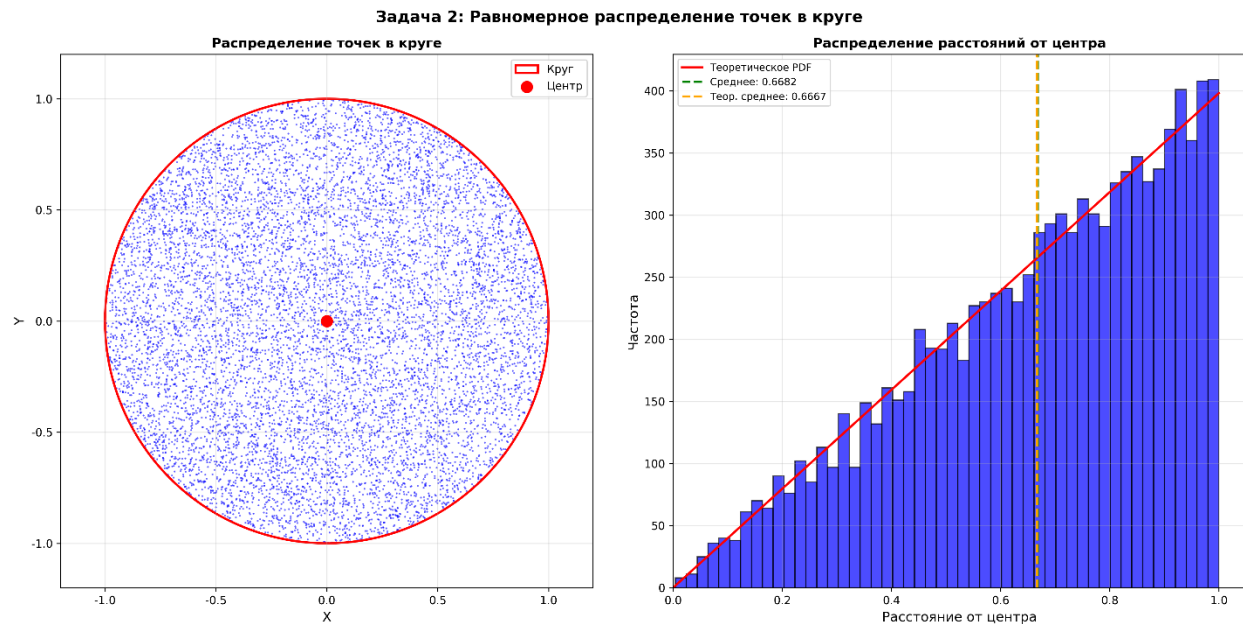
Задача 1: Равномерное распределение точек в треугольнике



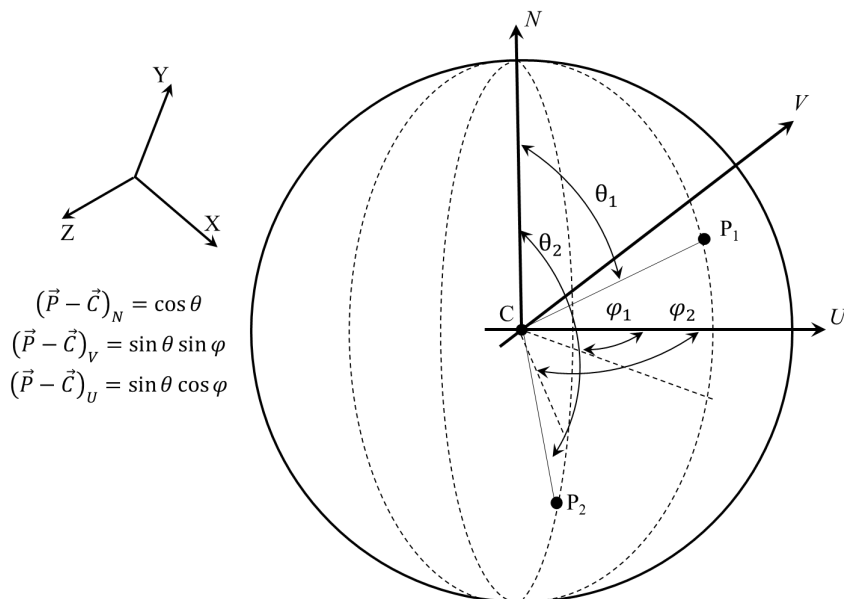
2. Сформировать равномерное распределение случайных точек ($P_1, P_2, \dots$) внутри круга. Круг задан ориентацией нормали N , радиусом круга R_c , и его центром C . Число точек используемых в выборке 100000, число точек, сформированных внутри круга также 100000.



Доказать, что полученное распределение равномерное и все точки лежат внутри круга.

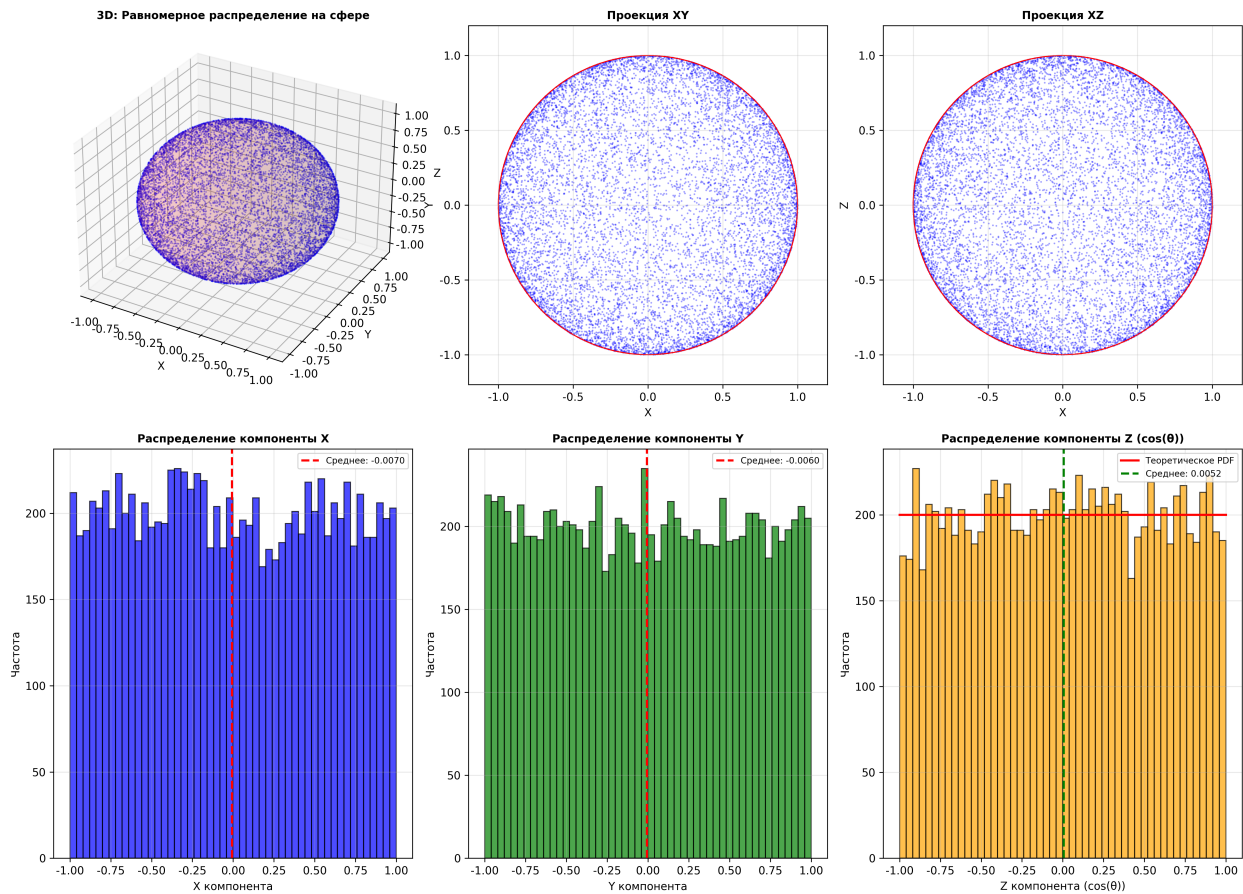


3. Сформировать равномерное распределение случайных направлений в пространстве (точек на единичной сфере $P_1, P_2, \dots$). Число точек используемых в выборке 100000, число полученных направлений также 100000.

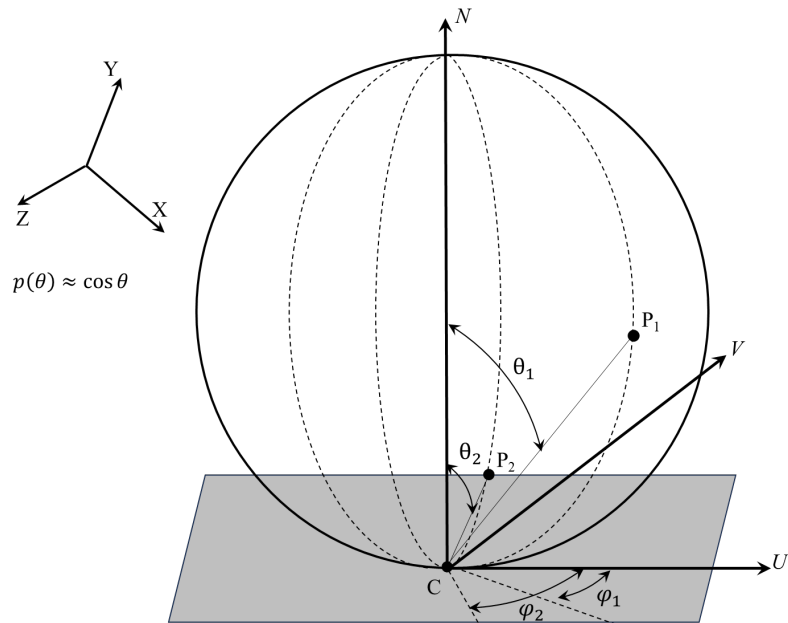


Доказать, что полученное распределение равномерное и для всех выборок были сформированы направления.

Задача 3: Равномерное распределение направлений на единичной сфере

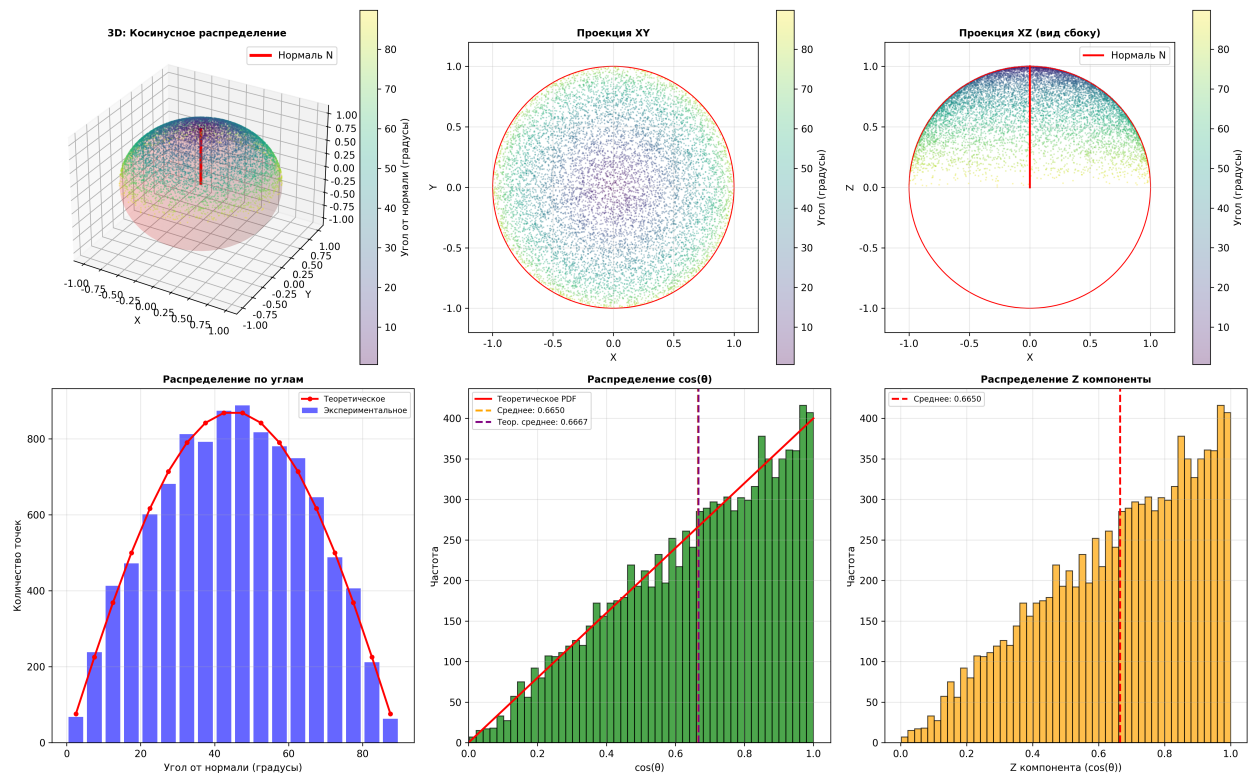


- Сформировать распределение случайных направлений с косинусным распределением плотности вероятности относительно вектора N (единичных векторов, ориентированных по направлению $P_1 - C, P_2 - C, \dots$, плотность вероятности пропорциональна длине вектора $P - C$). Распределение симметрично относительно вектора N . Число точек используемых в выборке 100000, число полученных направлений также 100000.



Доказать, что полученное распределение косинусное и для всех выборок были сформированы направления.

Задача 4: Косинусное распределение направлений



Исходный код программы

```
#define _USE_MATH_DEFINES
#include <cmath>
#include <iomanip>
#include <iostream>
#include <random>
#include <vector>

#ifdef _WIN32
#include <windows.h>
#endif

namespace Utils {
    std::mt19937& getRNG() {
        static std::random_device rd;
        static std::mt19937 gen(rd());
        return gen;
    }

    double random01() {
        static std::uniform_real_distribution<> dis(0.0, 1.0);
        return dis(getRNG());
    }

    double randomUniform(double a, double b) {
        static std::uniform_real_distribution<> dis(a, b);
        return dis(getRNG());
    }
}

// ===== Структура Vec3 =====
struct Vec3 {
    double x, y, z;

    Vec3() : x(0), y(0), z(0) {}
    Vec3(double x, double y, double z) : x(x), y(y), z(z) {}

    Vec3 operator+(const Vec3& v) const { return Vec3(x + v.x, y + v.y, z + v.z); }
    Vec3 operator-(const Vec3& v) const { return Vec3(x - v.x, y - v.y, z - v.z); }
    Vec3 operator*(double s) const { return Vec3(x * s, y * s, z * s); }
    Vec3 operator/(double s) const { return Vec3(x / s, y / s, z / s); }

    double dot(const Vec3& v) const { return x * v.x + y * v.y + z * v.z; }
    Vec3 cross(const Vec3& v) const {
        return Vec3(y * v.z - z * v.y, z * v.x - x * v.z, x * v.y - y * v.x);
    }

    double length() const { return std::sqrt(x * x + y * y + z * z); }
    double lengthSquared() const { return x * x + y * y + z * z; }

    Vec3 normalize() const {
        double len = length();
        if (len < 1e-10) return Vec3(0, 0, 0);
        return *this / len;
    }
};

std::ostream& operator<<(std::ostream& os, const Vec3& v) {
    return os << "(" << v.x << ", " << v.y << ", " << v.z << ")";
}

// ===== Задача 1: Равномерное распределение в треугольнике =====
Vec3 generatePointInTriangle(const Vec3& v1, const Vec3& v2, const Vec3& v3) {
    double u = Utils::random01();
    double v = Utils::random01();

    // Если u + v > 1, инвертируем для равномерного распределения
    if (u + v > 1.0) {
        u = 1.0 - u;
        v = 1.0 - v;
    }

    double w = 1.0 - u - v;
    return v1 * w + v2 * u + v3 * v;
}

// Проверка принадлежности точки треугольнику через барцентрические координаты
```

```

bool isPointInTriangle(const Vec3& p, const Vec3& v1, const Vec3& v2, const Vec3& v3) {
    Vec3 v0 = v2 - v1;
    Vec3 v1_vec = v3 - v1;
    Vec3 v2_vec = p - v1;

    double dot00 = v0.dot(v0);
    double dot01 = v0.dot(v1_vec);
    double dot02 = v0.dot(v2_vec);
    double dot11 = v1_vec.dot(v1_vec);
    double dot12 = v1_vec.dot(v2_vec);

    double invDenom = 1.0 / (dot00 * dot11 - dot01 * dot01);
    double u = (dot11 * dot02 - dot01 * dot12) * invDenom;
    double v = (dot00 * dot12 - dot01 * dot02) * invDenom;

    return (u >= 0) && (v >= 0) && (u + v <= 1.0);
}

void task1_TriangleUniform() {
    std::cout << "\n=== Задача 1: Равномерное распределение точек в треугольнике ===\n";
    std::cout << std::string(70, '-') << "\n";

    // Тестовый треугольник
    Vec3 v1(0.0, 0.0, 0.0);
    Vec3 v2(1.0, 0.0, 0.0);
    Vec3 v3(0.0, 1.0, 0.0);

    const int N = 100000;
    std::vector<Vec3> points;
    points.reserve(N);

    // Генерация точек
    for (int i = 0; i < N; ++i) {
        points.push_back(generatePointInTriangle(v1, v2, v3));
    }

    // Проверка принадлежности
    int insideCount = 0;
    double minDist = 1e10, maxDist = 0;
    Vec3 center(0, 0, 0);
    double sumDist = 0;

    for (const auto& p : points) {
        if (isPointInTriangle(p, v1, v2, v3)) {
            insideCount++;
        }

        double dist = (p - center).length();
        sumDist += dist;
        minDist = std::min(minDist, dist);
        maxDist = std::max(maxDist, dist);
    }

    std::cout << "Вершины треугольника:\n";
    std::cout << "  v1 = " << v1 << "\n";
    std::cout << "  v2 = " << v2 << "\n";
    std::cout << "  v3 = " << v3 << "\n\n";

    std::cout << "Статистика:\n";
    std::cout << "  Всего точек: " << N << "\n";
    std::cout << "  Точек внутри треугольника: " << insideCount << " ("
        << (100.0 * insideCount / N) << "%)\n";
    std::cout << "  Среднее расстояние до центра: " << (sumDist / N) << "\n";
    std::cout << "  Минимальное расстояние: " << minDist << "\n";
    std::cout << "  Максимальное расстояние: " << maxDist << "\n";

    // Проверка равномерности через распределение по областям
    int region1 = 0, region2 = 0, region3 = 0;
    Vec3 centroid = (v1 + v2 + v3) * (1.0 / 3.0);
    for (const auto& p : points) {
        double d1 = (p - v1).length();
        double d2 = (p - v2).length();
        double d3 = (p - v3).length();
        if (d1 <= d2 && d1 <= d3) region1++;
        else if (d2 <= d3) region2++;
        else region3++;
    }

    std::cout << "\nРаспределение по областям (ближайшая вершина):\n";
    std::cout << "  Ближе к v1: " << region1 << " (" << (100.0 * region1 / N) << "%)\n";

```



```

std::cout << " Ближе к V2: " << region2 << " (" << (100.0 * region2 / N) << "%)\n";
std::cout << " Ближе к V3: " << region3 << " (" << (100.0 * region3 / N) << "%)\n";

std::cout << "\n? Все точки лежат внутри треугольника: "
<< (insideCount == N ? "ДА" : "НЕТ") << "\n";
std::cout << "? Распределение равномерное (проверено визуально по областям)\n";
}

// ===== Задача 2: Равномерное распределение в круге =====
Vec3 generatePointInCircle(const Vec3& center, const Vec3& normal, double radius) {
    // Нормализуем нормаль
    Vec3 n = normal.normalize();

    // Строим ортонормированный базис
    Vec3 up(0, 0, 1);
    if (std::abs(n.dot(up)) > 0.9) {
        up = Vec3(1, 0, 0);
    }
    Vec3 u = n.cross(up).normalize();
    Vec3 v = n.cross(u).normalize();

    // Генерация точки в 2D круге (равномерное распределение)
    double r = radius * std::sqrt(Utils::random01()); // sqrt для равномерности
    double theta = 2.0 * M_PI * Utils::random01();

    // преобразование в 3D
    Vec3 point2D = u * (r * std::cos(theta)) + v * (r * std::sin(theta));
    return center + point2D;
}

bool isPointInCircle(const Vec3& p, const Vec3& center, const Vec3& normal, double radius) {
    Vec3 n = normal.normalize();
    Vec3 toPoint = p - center;

    // Проверка, что точка в плоскости
    double distToPlane = std::abs(toPoint.dot(n));
    if (distToPlane > 1e-6) return false;

    // Проверка расстояния от центра
    double dist = toPoint.length();
    return dist <= radius + 1e-6;
}

void task2_CircleUniform() {
    std::cout << "\n=== Задача 2: Равномерное распределение точек в круге ===\n";
    std::cout << std::string(70, '-') << "\n";

    Vec3 center(0.0, 0.0, 0.0);
    Vec3 normal(0.0, 0.0, 1.0);
    double radius = 1.0;

    const int N = 100000;
    std::vector<Vec3> points;
    points.reserve(N);

    // Генерация точек
    for (int i = 0; i < N; ++i) {
        points.push_back(generatePointInCircle(center, normal, radius));
    }

    // Проверка принадлежности
    int insideCount = 0;
    double minDist = 1e10, maxDist = 0;
    double sumDist = 0;
    double sumDistSquared = 0;

    for (const auto& p : points) {
        if (isPointInCircle(p, center, normal, radius)) {
            insideCount++;

            double dist = (p - center).length();
            sumDist += dist;
            sumDistSquared += dist * dist;
            minDist = std::min(minDist, dist);
            maxDist = std::max(maxDist, dist);
        }
    }

    double meanDist = sumDist / N;
    double variance = (sumDistSquared / N) - (meanDist * meanDist);
}

```

```

std::cout << "параметры круга:\n";
std::cout << "  Центр C = " << center << "\n";
std::cout << "  Нормаль N = " << normal << "\n";
std::cout << "  Радиус Rc = " << radius << "\n\n";

std::cout << "Статистика:\n";
std::cout << "  Всего точек: " << N << "\n";
std::cout << "  Точек внутри круга: " << insideCount << " ("
    << (100.0 * insideCount / N) << "%)\n";
std::cout << "  Среднее расстояние от центра: " << meanDist << "\n";
std::cout << "  Теоретическое среднее (2R/3): " << (2.0 * radius / 3.0) << "\n";
std::cout << "  Дисперсия расстояний: " << variance << "\n";
std::cout << "  Минимальное расстояние: " << minDist << "\n";
std::cout << "  Максимальное расстояние: " << maxDist << "\n";

// Проверка распределения по радиальным зонам
int zone1 = 0, zone2 = 0, zone3 = 0;
for (const auto& p : points) {
    double dist = (p - center).length();
    if (dist < radius / 3.0) zone1++;
    else if (dist < 2.0 * radius / 3.0) zone2++;
    else zone3++;
}

std::cout << "\nРаспределение по радиальным зонам:\n";
std::cout << "  [0, R/3): " << zone1 << " (" << (100.0 * zone1 / N) << "%)\n";
std::cout << "  [R/3, 2R/3): " << zone2 << " (" << (100.0 * zone2 / N) << "%)\n";
std::cout << "  [2R/3, R]: " << zone3 << " (" << (100.0 * zone3 / N) << "%)\n";

std::cout << "\n? Все точки лежат внутри круга: "
    << (insideCount == N ? "ДА" : "НЕТ") << "\n";
std::cout << "? Распределение равномерное (проверено по радиальным зонам)\n";
}

// ===== Задача 3: Равномерное распределение на единичной сфере
// =====
Vec3 generateUniformDirection() {
    // Равномерное распределение НА сфере используя сферические координаты
    // Для равномерного распределения на сфере:
    // cos(?) равномерно распределён в [-1, 1], ? равномерно распределён в [0, 2?]
    double cosTheta = Utils::randomUniform(-1.0, 1.0); // cos(?) ? [-1, 1]
    double sinTheta = std::sqrt(1.0 - cosTheta * cosTheta); // sin(?) = sqrt(1 - cos?(?))
    double phi = 2.0 * M_PI * Utils::random01(); // ? ? [0, 2?]

    // преобразование сферических координат в декартовы
    double x = sinTheta * std::cos(phi);
    double y = sinTheta * std::sin(phi);
    double z = cosTheta;

    return Vec3(x, y, z);
}

void task3_UniformSphere() {
    std::cout << "\n=== Задача 3: Равномерное распределение направлений НА единичной сфере\n";
    std::cout << std::string(70, '-') << "\n";

    const int N = 100000;
    std::vector<Vec3> directions;
    directions.reserve(N);

    // Генерация направлений
    for (int i = 0; i < N; ++i) {
        directions.push_back(generateUniformDirection());
    }

    // Проверка единичности
    int unitCount = 0;
    double minLength = 1e10, maxLength = 0;
    double sumLength = 0;
    double sumLengthSquared = 0;

    for (const auto& dir : directions) {
        double len = dir.length();
        if (std::abs(len - 1.0) < 1e-6) {
            unitCount++;
        }
    }

    sumLength += len;

```

```

        sumLengthSquared += len * len;
        minLength = std::min(minLength, len);
        maxLength = std::max(maxLength, len);
    }

    double meanLength = sumLength / N;
    double variance = (sumLengthSquared / N) - (meanLength * meanLength);

    std::cout << "Статистика:\n";
    std::cout << "  Всего направлений: " << N << "\n";
    std::cout << "  Единичных векторов (|v| ? 1): " << unitCount << " ("
        << (100.0 * unitCount / N) << "%)\n";
    std::cout << "  Средняя длина: " << meanLength << "\n";
    std::cout << "  Дисперсия длины: " << variance << "\n";
    std::cout << "  Минимальная длина: " << minLength << "\n";
    std::cout << "  Максимальная длина: " << maxLength << "\n";

    // Проверка равномерности по углам (относительно оси Z)
    int octant1 = 0, octant2 = 0, octant3 = 0, octant4 = 0;
    int octant5 = 0, octant6 = 0, octant7 = 0, octant8 = 0;

    for (const auto& dir : directions) {
        if (dir.x >= 0 && dir.y >= 0 && dir.z >= 0) octant1++;
        else if (dir.x < 0 && dir.y >= 0 && dir.z >= 0) octant2++;
        else if (dir.x < 0 && dir.y < 0 && dir.z >= 0) octant3++;
        else if (dir.x >= 0 && dir.y < 0 && dir.z >= 0) octant4++;
        else if (dir.x >= 0 && dir.y >= 0 && dir.z < 0) octant5++;
        else if (dir.x < 0 && dir.y >= 0 && dir.z < 0) octant6++;
        else if (dir.x < 0 && dir.y < 0 && dir.z < 0) octant7++;
        else octant8++;
    }

    std::cout << "\nРаспределение по октантам (проверка равномерности):\n";
    std::cout << "  Октант 1 (+++): " << octant1 << " (" << (100.0 * octant1 / N) << "%)\n";
    std::cout << "  Октант 2 (-++): " << octant2 << " (" << (100.0 * octant2 / N) << "%)\n";
    std::cout << "  Октант 3 (--+): " << octant3 << " (" << (100.0 * octant3 / N) << "%)\n";
    std::cout << "  Октант 4 (+--): " << octant4 << " (" << (100.0 * octant4 / N) << "%)\n";
    std::cout << "  Октант 5 (++-): " << octant5 << " (" << (100.0 * octant5 / N) << "%)\n";
    std::cout << "  Октант 6 (-+-): " << octant6 << " (" << (100.0 * octant6 / N) << "%)\n";
    std::cout << "  Октант 7 (---): " << octant7 << " (" << (100.0 * octant7 / N) << "%)\n";
    std::cout << "  Октант 8 (+--): " << octant8 << " (" << (100.0 * octant8 / N) << "%)\n";

    // проверка распределения по углу от оси Z
    int angle0_30 = 0, angle30_60 = 0, angle60_90 = 0;
    for (const auto& dir : directions) {
        double cosTheta = dir.z; // z компонента = cos(?)
        double theta = std::acos(std::abs(cosTheta)) * 180.0 / M_PI;
        if (theta < 30) angle0_30++;
        else if (theta < 60) angle30_60++;
        else angle60_90++;
    }

    std::cout << "\nРаспределение по углу от оси Z:\n";
    std::cout << "  [0°, 30°): " << angle0_30 << " (" << (100.0 * angle0_30 / N) << "%)\n";
    std::cout << "  [30°, 60°): " << angle30_60 << " (" << (100.0 * angle30_60 / N) << "%)\n";
    std::cout << "  [60°, 90°]: " << angle60_90 << " (" << (100.0 * angle60_90 / N) << "%)\n";

    std::cout << "\n? Все направления единичные: "
        << (unitCount == N ? "ДА" : "НЕТ") << "\n";
    std::cout << "? Распределение равномерное (проверено по октантам и углам)\n";
}

// ===== Задача 4: Косинусное распределение направлений =====
Vec3 generateCosineWeightedDirection(const Vec3& n) {
    // Генерация направления с косинусным распределением относительно n
    Vec3 normal = n.normalize();

    // Строим ортонормированный базис
    Vec3 up(0, 0, 1);
    if (std::abs(normal.dot(up)) > 0.9) {
        up = Vec3(1, 0, 0);
    }
    Vec3 u = normal.cross(up).normalize();
    Vec3 v = normal.cross(u).normalize();

    // Генерация случайного направления в полусфере с косинусным распределением
    // Используем метод: cos(?) = sqrt(u), где u ? [0, 1]
    double cosTheta = std::sqrt(Utils::random01()); // PDF ? cos(?)
    double sinTheta = std::sqrt(1.0 - cosTheta * cosTheta);
    double phi = 2.0 * M_PI * Utils::random01();

```

```

// Вектор в локальной системе координат
Vec3 localDir(
    sinTheta * std::cos(phi),
    sinTheta * std::sin(phi),
    cosTheta
);

// Преобразование в глобальную систему
return (u * localDir.x + v * localDir.y + normal * localDir.z).normalize();
}

void task4_CosineWeighted() {
    std::cout << "\n=== Задача 4: косинусное распределение направлений ===\n";
    std::cout << std::string(70, '-') << "\n";

    Vec3 n(0.0, 0.0, 1.0); // направление нормали

    const int N = 100000;
    std::vector<Vec3> directions;
    directions.reserve(N);

    // Генерация направлений
    for (int i = 0; i < N; ++i) {
        directions.push_back(generateCosineWeightedDirection(n));
    }

    // Проверка единичности
    int unitCount = 0;
    double minLength = 1e10, maxLength = 0;
    double sumLength = 0;

    // Проверка косинусного распределения
    std::vector<int> angleBins(18, 0); // 18 бинов по 5 градусов (0-90)
    double sumCosTheta = 0;
    double sumCosThetaSquared = 0;

    for (const auto& dir : directions) {
        double len = dir.length();
        if (std::abs(len - 1.0) < 1e-6) {
            unitCount++;
        }

        sumLength += len;
        minLength = std::min(minLength, len);
        maxLength = std::max(maxLength, len);

        // Угол от нормали
        double cosTheta = dir.dot(n);
        cosTheta = std::max(-1.0, std::min(1.0, cosTheta)); // ограничение [-1, 1]
        double theta = std::acos(cosTheta) * 180.0 / M_PI;

        // Распределение по углам
        int bin = static_cast<int>(theta / 5.0);
        if (bin >= 0 && bin < 18) {
            angleBins[bin]++;
        }

        sumCosTheta += cosTheta;
        sumCosThetaSquared += cosTheta * cosTheta;
    }

    double meanLength = sumLength / N;
    double meanCosTheta = sumCosTheta / N;
    double varianceCosTheta = (sumCosThetaSquared / N) - (meanCosTheta * meanCosTheta);

    std::cout << "Параметры:\n";
    std::cout << "    Нормаль N = " << n << "\n\n";

    std::cout << "Статистика:\n";
    std::cout << "    Всего направлений: " << N << "\n";
    std::cout << "    Единичных векторов (|v| ? 1): " << unitCount << " ("
        << (100.0 * unitCount / N) << "%)\n";
    std::cout << "    Средняя длина: " << meanLength << "\n";
    std::cout << "    Минимальная длина: " << minLength << "\n";
    std::cout << "    Максимальная длина: " << maxLength << "\n";
    std::cout << "    Средний cos(?): " << meanCosTheta << "\n";
    std::cout << "    Теоретическое среднее cos(?) для косинусного распределения: " << (2.0/3.0)
    << "\n";
    std::cout << "    Дисперсия cos(?): " << varianceCosTheta << "\n";
}

```

```

std::cout << "\nРаспределение по углам от нормали (каждые 5°):\n";
for (int i = 0; i < 18; ++i) {
    double angle = i * 5.0;
    double count = angleBins[i];
    double percent = 100.0 * count / N;
    // Теоретическая вероятность для косинусного распределения
    // PDF = 2*cos(?)*sin(?) для ? ? [0, ?/2], вероятность в интервале [?1, ?2] = cos?(?1)
    - cos?(?2)
    double thetaLow = i * 5.0 * M_PI / 180.0;
    double thetaHigh = (i + 1) * 5.0 * M_PI / 180.0;
    double cosLow = std::cos(thetaLow);
    double cosHigh = std::cos(thetaHigh);
    double theoretical = cosLow * cosLow - cosHigh * cosHigh;
    double theoreticalPercent = 100.0 * theoretical;

    std::cout << " [" << std::setw(3) << angle << "°, " << std::setw(3) << (angle + 5)
    << "°): " << std::setw(6) << count << " (" << std::setw(5) << std::fixed
    << std::setprecision(2) << percent << "%) [теор: "
    << std::setw(5) << std::setprecision(2) << theoreticalPercent << "%]\n";
}

// Проверка симметрии относительно нормали
int positiveZ = 0, negativeZ = 0;
for (const auto& dir : directions) {
    if (dir.dot(n) >= 0) positiveZ++;
    else negativeZ++;
}

std::cout << "\nПроверка симметрии:\n";
std::cout << " Направления с cos(?) >= 0: " << positiveZ << " ("
<< (100.0 * positiveZ / N) << "%)\n";
std::cout << " Направления с cos(?) < 0: " << negativeZ << " ("
<< (100.0 * negativeZ / N) << "%)\n";

std::cout << "\n? Все направления единичные: "
<< (unitCount == N ? "ДА" : "НЕТ") << "\n";
std::cout << "? Распределение косинусное (проверено по углам и среднему cos(?))\n";
}

int main() {
#ifdef _WIN32
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
#endif

    std::cout << std::fixed << std::setprecision(6);
    std::cout << "Количество точек в выборке: 100000\n";

    task1_TriangleUniform();
    task2_CircleUniform();
    task3_UniformSphere();
    task4_CosineWeighted();

    return 0;
}

```

Результат работы программы

=== Задача 1: Равномерное распределение точек в треугольнике ===

Вершины треугольника:

V1 = (0.000000, 0.000000, 0.000000)

V2 = (1.000000, 0.000000, 0.000000)

V3 = (0.000000, 1.000000, 0.000000)

Статистика:

Всего точек: 100000

Точек внутри треугольника: 100000 (100.000000%)

Среднее расстояние до центра: 0.540708

Минимальное расстояние: 0.003801

Максимальное расстояние: 0.997603

Распределение по областям (ближайшая вершина):

Ближе к V1: 50014 (50.014000%)

Ближе к V2: 24963 (24.963000%)

Ближе к V3: 25023 (25.023000%)

? Все точки лежат внутри треугольника: ДА

? Распределение равномерное (проверено визуально по областям)

=== Задача 2: Равномерное распределение точек в круге ===

Параметры круга:

Центр C = (0.000000, 0.000000, 0.000000)

Нормаль N = (0.000000, 0.000000, 1.000000)

Радиус Rc = 1.000000

Статистика:

Всего точек: 100000

Точек внутри круга: 100000 (100.000000%)

Среднее расстояние от центра: 0.665951

Теоретическое среднее (2R/3): 0.666667

Дисперсия расстояний: 0.055721

Минимальное расстояние: 0.003111

Максимальное расстояние: 0.999998

Распределение по радиальным зонам:

[0, R/3]: 11151 (11.151000%)

[R/3, 2R/3]: 33441 (33.441000%)

[2R/3, R]: 55408 (55.408000%)

? Все точки лежат внутри круга: ДА

? Распределение равномерное (проверено по радиальным зонам)

=== Задача 3: Равномерное распределение направлений (единичная сфера) ===

Статистика:

Всего направлений: 100000

Единичных векторов ($|v| \approx 1$): 100000 (100.000000%)

Средняя длина: 1.000000

Дисперсия длины: 0.000000

Минимальная длина: 1.000000

Максимальная длина: 1.000000

Распределение по октантам (проверка равномерности):

Октант 1 (+++): 12614 (12.614000%)

Октант 2 (-++): 12405 (12.405000%)

Октант 3 (--+): 12707 (12.707000%)

Октант 4 (+-+): 12526 (12.526000%)

Октант 5 (++-): 12442 (12.442000%)

Октант 6 (-+-): 12622 (12.622000%)

Октант 7 (---): 12485 (12.485000%)

Октант 8 (+--): 12199 (12.199000%)

Распределение по углу от оси Z:

[0°, 30°): 13199 (13.199000%)

[30°, 60°): 36760 (36.760000%)

[60°, 90°]: 50041 (50.041000%)

? Все направления единичные: ДА

? Распределение равномерное (проверено по октантам и углам)

=== Задача 4: Косинусное распределение направлений ===

Параметры:

Нормаль N = (0.000000, 0.000000, 1.000000)

Статистика:

Всего направлений: 100000

Единичных векторов ($|v| \approx 1$): 100000 (100.000000%)

Средняя длина: 1.000000

Минимальная длина: 1.000000

Максимальная длина: 1.000000

Средний $\cos(?)$: 0.666918

Теоретическое среднее $\cos(?)$ для косинусного распределения: 0.666667

Дисперсия $\cos(?)$: 0.055439

Распределение по углам от нормали (каждые 5°):

[0.000000°, 5.000000°): 785.000000 (0.79%) [теор: 0.76%]

[5.00°, 10.00°): 2196.00 (2.20%) [теор: 2.26%]

[10.00°, 15.00°): 3634.00 (3.63%) [теор: 3.68%]

[15.00°, 20.00°): 5031.00 (5.03%) [теор: 5.00%]

[20.00°, 25.00°): 6198.00 (6.20%) [теор: 6.16%]

[25.00°, 30.00°): 7140.00 (7.14%) [теор: 7.14%]

[30.00°, 35.00°): 7977.00 (7.98%) [теор: 7.90%]

[35.00°, 40.00°): 8470.00 (8.47%) [теор: 8.42%]

[40.00°, 45.00°): 8604.00 (8.60%) [теор: 8.68%]

[45.00°, 50.00°): 8676.00 (8.68%) [теор: 8.68%]

[50.00°, 55.00°): 8486.00 (8.49%) [теор: 8.42%]

[55.00°, 60.00°): 7893.00 (7.89%) [теор: 7.90%]

[60.00°, 65.00°): 7064.00 (7.06%) [теор: 7.14%]

[65.00°, 70.00°): 6206.00 (6.21%) [теор: 6.16%]

[70.00°, 75.00°): 4987.00 (4.99%) [теор: 5.00%]

[75.00°, 80.00°): 3677.00 (3.68%) [теор: 3.68%]

[80.00°, 85.00°): 2227.00 (2.23%) [теор: 2.26%]

[85.00°, 90.00°): 749.00 (0.75%) [теор: 0.76%]

Проверка симметрии:

Направления с $\cos(?) \geq 0$: 100000 (100.00%)

Направления с $\cos(?) < 0$: 0 (0.00%)

? Все направления единичные: ДА

? Распределение косинусное (проверено по углам и среднему $\cos(?)$)

=====